

PS159 — Person A Procedure

Analysis Engine — PCAP to JSON, Start to Finish

SecureMailScope

SIH MVP — 2-Day Build

0. Your Job, in One Sentence
1. The Contract You're Building Toward
2. Environment Setup (Do This First, ~20 Minutes)
3. Day 1
4. Day 2
5. Fallback Rules — Yours
6. Handoff Checklist — What Person B Needs From You
7. Judge Questions About Your Half — Know These Cold

0. Your Job, in One Sentence

You take a PCAP file and produce `analysis_result.json`. That file is the *only* thing Person B ever sees from you. You never open their dashboard code; they never open yours. If you find yourself needing to know how the dashboard works, stop — that means the JSON contract is missing something, and that's a five-minute conversation, not a reason to cross the wall.

Mindset while you build this: every field you extract is something an attacker would actually check for. An old TLS version or a weak cipher (RC4, DES, export-grade, anything MD5-signed) means recorded traffic can be decrypted later even without breaking anything live. A certificate that's expired or doesn't chain properly is exactly what lets a fake server impersonate a real one. STARTTLS done wrong — client asks to upgrade, upgrade silently fails, client just keeps talking in plaintext — is a live downgrade attack, not a theoretical one. You're not filling in a schema for its own sake; you're building the thing that catches a mail server that looks fine until someone actually tries these tricks on it.

1. The Contract You're Building Toward

Agree this with Person B in the first 30 minutes, before you write a line of extraction code. Everything after this section is about how you produce something that matches this shape.

```
{
  "metadata": {
    "filename": "weak.pcap",
    "analyzed_at": "2026-09-27T10:00:00Z"
  },
  "summary": {
    "total_sessions": 3,
    "low_risk": 1,
    "medium_risk": 0,
    "high_risk": 2
  },
  "sessions": [
    {
      "session_id": "session_001",
      "protocol": "SMTP",
      "tls": {
        "detected": true,
        "version": "TLS 1.2",
        "cipher_suite": "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256",
        "key_exchange": "ECDHE"
      },
      "certificate": {
        "present": true,
        "subject": "CN=mail.example.com",
        "issuer": "CN=Test CA",
        "valid_from": "2025-01-01",
        "valid_until": "2026-01-01",
        "expired": false
      },
      "starttls": {
        "attempted": true,
        "succeeded": true,
        "note": null
      },
      "findings": [
        {
          "title": "Modern TLS and valid certificate",
          "severity": "LOW",
          "description": "Session uses TLS 1.2 with a strong cipher suite and a valid certificate.",
          "evidence": "tcp.stream==4",
          "recommendation": "No action needed."
        }
      ],
      "risk": { "level": "LOW", "score": 10 }
    }
  ]
}
```

Rule that overrides everything else in this document: if you can't reliably extract a field, it's `null`. Never invented. A judge asking "why does this say unknown?" is a fine conversation. A judge catching a fabricated cipher suite ends your demo.

2. Environment Setup (Do This First, ~20 Minutes)

```
# tshark
sudo apt install tshark      # Linux
brew install wireshark      # Mac – installs tshark too

# confirm your version and check exact field names –
# these vary slightly across tshark versions, and you will
# need this output later, so run it now and keep it open
tshark -v
tshark -G fields | grep -i x509

python3 -m venv venv
source venv/bin/activate
pip install cryptography
```

requirements.txt (your half):

```
cryptography
```

Repo folders that are yours:

```
PS159-MVP/
├─ analyzer/
│   ├── pcap_parser.py
│   ├── tls_analyzer.py
│   ├── certificate_analyzer.py
│   ├── starttls_analyzer.py
│   ├── security_rules.py
│   └─ analyzer.py
├─ data/          # your test PCAPs
└─ sample_results/ # your JSON output – the only folder Person B reads from
```

Git: work on a branch called `analyzer`. Commit freely, as often as you like — nothing here can break the dashboard, because the dashboard never touches your code, only your output files.

```
git checkout -b analyzer
git commit -m "analyzer: <what you just did>"
```

3. Day 1

Step 1 — Source Test PCAPs (30-60 min)

Do not build your own mail server for this — that’s a half-day sink you don’t have in a 2-day build. Pull real traffic instead:

- Search “Wireshark SampleCaptures SMTP” and “Wireshark SampleCaptures TLS” — Wireshark’s own site hosts real sample captures.
- Search “Netresec public PCAP repository” for broader traffic, including weaker/older TLS examples.

Get at least two files: one clean modern-TLS session, one weak/old-TLS or plaintext session. Rename them `data/good.pcap` and `data/weak.pcap`. If you don’t find something perfect in 20 minutes, stop looking and use whatever real SMTP/IMAP/TLS pcap you can get — proving the pipeline runs end-to-end on real bytes matters more than a perfect dataset.

Step 2 — Identify Sessions (30-60 min)

```
tshark -r data/good.pcap -Y "tcp.port==25 || tcp.port==587 || tcp.port==143 || tcp.port==993 || tcp.port==110 || tcp.port==995" \
-T fields -e tcp.stream -e tcp.port -e ip.src -e ip.dst -E header=y > sessions.csv
```

Map port to protocol yourself in Python:

```
PORT_TO_PROTOCOL = {
    25: "SMTP", 587: "SMTP",
    143: "IMAP", 993: "IMAP",
    110: "POP3", 995: "POP3",
}
```

Each unique `tcp.stream` value is one session. This becomes your `session_id` (e.g. `session_001` for stream `0`, etc. — pick a consistent mapping and stick to it).

Step 3 — Extract TLS Handshake Fields (1-2 hrs)

```
tshark -r data/good.pcap -Y "tls.handshake.type==1 || tls.handshake.type==2" \
-T fields -e tcp.stream -e tls.handshake.type -e tls.handshake.version -e tls.handshake.ciphersuite \
-E header=y -E separator=, > tls_handshakes.csv
```

`tls.handshake.type==1` is ClientHello, `==2` is ServerHello. **The ServerHello row is the one you want** — it carries the version and cipher suite the two sides actually *negotiated*, not just what the client offered.

```
def extract_tls(stream_id, tls_rows):
    server_hello = [r for r in tls_rows if r["tcp.stream"] == stream_id and r["tls.handshake.type"] == "2"]
    if not server_hello:
        return {"detected": False, "version": None, "cipher_suite": None, "key_exchange": None}
    row = server_hello[0]
    return {
        "detected": True,
        "version": row["tls.handshake.version"],
        "cipher_suite": row["tls.handshake.ciphersuite"],
        "key_exchange": None, # derive from cipher suite name if time allows, else leave null
    }
```

Step 4 — Extract Certificate Fields (1 hr)

```
tshark -r data/good.pcap -Y "tls.handshake.certificate" \
-T fields -e tcp.stream -e x509af.notBefore -e x509af.notAfter -e x509if.printableString \
-E header=y -E separator=, > certs.csv
```

These field names are the most likely thing to break on your specific tshark version. Confirm with the `tshark -G fields | grep -i x509` output from Step 0 before trusting them. If they don't line up:

```
tshark -r data/good.pcap -Y "tls.handshake.certificate" -V
```

This dumps the full decoded structure so you can find the correct field names by hand. Budget real time for this — it's the step most likely to eat your schedule.

```
from datetime import datetime

def check_expiry(not_after_str):
    if not not_after_str:
        return None
    expiry = datetime.strptime(not_after_str, "%b %d %H:%M:%S %Y GMT") # adjust to your actual format
    return expiry < datetime.utcnow()
```

Step 5 — Write `security_rules.py`

```
def grade_tls(tls):
    if not tls or not tls.get("detected"):
        return "HIGH", "No TLS detected — session in plaintext"
    if tls["version"] in ["SSLv3", "TLS 1.0", "TLS 1.1"]:
        return "HIGH", f"Deprecated TLS version: {tls['version']}"
    weak_markers = ["RC4", "DES", "EXPORT", "NULL", "MD5"]
    if tls.get("cipher_suite") and any(w in tls["cipher_suite"] for w in weak_markers):
        return "HIGH", f"Weak cipher suite: {tls['cipher_suite']}"
    return "LOW", "Modern TLS version and cipher"

def grade_cert(cert):
    if not cert or not cert.get("present"):
        return "MEDIUM", "No certificate found in handshake"
    if cert.get("expired"):
        return "HIGH", "Certificate expired"
    return "LOW", "Certificate present and valid"

def overall_risk(tls_level, cert_level):
    levels = {"LOW": 1, "MEDIUM": 2, "HIGH": 3}
    worst = max(tls_level, cert_level, key=lambda l: levels[l])
    scores = {"LOW": 15, "MEDIUM": 50, "HIGH": 80}
    return worst, scores[worst]
```

Step 6 — Wire It Together in `analyzer.py`

```
import json
import sys
from datetime import datetime, timezone

def analyze_pcap(pcap_path):
    # 1. run tshark subprocess calls from Steps 2-4
    # 2. group into sessions
    # 3. grade each session with security_rules.py
    # 4. assemble into the schema from Section 1
    sessions = [] # fill this in
```

```

summary = {
    "total_sessions": len(sessions),
    "low_risk": sum(1 for s in sessions if s["risk"]["level"] == "LOW"),
    "medium_risk": sum(1 for s in sessions if s["risk"]["level"] == "MEDIUM"),
    "high_risk": sum(1 for s in sessions if s["risk"]["level"] == "HIGH"),
}
return {
    "metadata": {
        "filename": pcap_path,
        "analyzed_at": datetime.now(timezone.utc).isoformat(),
    },
    "summary": summary,
    "sessions": sessions,
}

if __name__ == "__main__":
    result = analyze_pcap(sys.argv[1])
    print(json.dumps(result, indent=2))

```

Make this runnable both as a library (`from analyzer import analyze_pcap`) and from the command line (`python analyzer.py data/good.pcap > sample_results/good.json`) — Person B will want the CLI form for their subprocess integration on Day 2.

Day 1 — Your Milestone

Run `analyzer.py` on `good.pcap`, produce one real `analysis_result.json` matching the schema, even if several fields are `null`. **Commit it, push your analyzer branch, and hand the file to Person B before you end the day.** This is the single most important checkpoint of Day 1 — everything on Day 2 assumes it happened.

4. Day 2

Step 1 — Run the Same Pipeline on `weak.pcap`

You should see visibly different output: older TLS version, weak cipher, or no TLS at all. **If `good.json` and `weak.json` look basically the same, stop — that's a bug in your extraction or a problem with your test data, not evidence the servers are equally secure.** Debug this before doing anything else today.

Step 2 — Attempt STARTTLS Detection (Stretch Goal — Do Not Let This Eat Your Day)

Only start this once Step 1 above is solid. Method:

1. On the plaintext portion of a session (before any TLS handshake begins), look for the client sending the literal command `STARTTLS` (SMTP) or `a1 STARTTLS` style (IMAP).
2. Check the server's response — a positive reply (e.g. `220` for SMTP) means the upgrade was accepted.
3. Check whether the *next* bytes on that same `tcp.stream` are TLS handshake records (`tls.record.content_type`) or are still readable plaintext. If they're still plaintext after a positive STARTTLS response, that's a silent downgrade — flag it `HIGH / CRITICAL`.

```
def check_starttls(stream_id, plaintext_lines, tls_start_offset):
    attempted = any("STARTTLS" in line.upper() for line in plaintext_lines)
    if not attempted:
        return {"attempted": False, "succeeded": None, "note": None}
    if tls_start_offset is not None:
        return {"attempted": True, "succeeded": True, "note": None}
    return {"attempted": True, "succeeded": False, "note": "STARTTLS offered but session continued in plaintext"}
```

Hard rule: if this isn't reliably working by early afternoon, set `starttls.note` = "Not available in this capture" on every session and move on. This was always a stretch goal, not a milestone. Do not let it threaten the rest of Day 2.

Step 3 — Final Cleanup Pass

- Walk through every field in the schema and confirm: is it `null` when genuinely unavailable, and populated when it is? No placeholders, no guesses.
- Produce final `sample_results/good.json`, `weak.json`, and `critical.json` (if you sourced a third distinct sample) from real PCAPs.
- Make sure `analyzer.py` runs cleanly from a fresh terminal (`python analyzer.py <path>`) — Person B needs this exact command to work for their live-upload integration.

Step 4 — Git Sync

```
git add sample_results/ analyzer/
git commit -m "analyzer: final JSON output for good/weak/critical, STARTTLS status noted"
git push origin analyzer
```

Merge to `main` once Person B confirms the dashboard renders your files with zero code changes on their end.

5. Fallback Rules — Yours

If this happens	Do this
A field can't be reliably extracted	<code>null</code> . Never invent it.
STARTTLS detection isn't working	<code>starttls.note = "Not available in this capture"</code> , move on
tshark field names don't match this document	<code>tshark -G fields \</code> <code>grep <keyword></code> and <code>-V full</code> decode to find real names for your version
<code>good.json</code> and <code>weak.json</code> look identical	Stop. Debug extraction or test data before continuing.
You're behind schedule on Day 2 afternoon	Cut STARTTLS and <code>critical.pcap</code> first. Never cut: real JSON for <code>good.pcap</code> and <code>weak.pcap</code> , or the null-not- invented rule.

6. Handoff Checklist — What Person B Needs From You

- ☐ `sample_results/good.json` and `weak.json`, real output, schema-correct
- ☐ `analyzer.py` runnable from the command line with one argument (the pcap path)
- ☐ Every unavailable field is `null`, none are fabricated
- ☐ You've told Person B, explicitly, whether STARTTLS detection is working or not
- ☐ Your `analyzer` branch is pushed and Person B can pull it without needing to read your code

7. Judge Questions About Your Half — Know These Cold

- **“Why tshark instead of writing your own packet parser?”** Because Wireshark’s dissectors for SMTP/IMAP/POP3/TLS are mature, actively maintained, and handle real-world edge cases we don’t have time to rediscover in 2 days. Reinventing that would be wasted effort, not rigor.
- **“How do you know a session is actually weak, not just different?”** The grading is rule-based against published standards — deprecated TLS versions, known-weak cipher markers (RC4, DES, export-grade, MD5), and certificate expiry are all objective checks, not opinions.
- **“What if a field genuinely can’t be extracted?”** It’s reported as `null` / unknown, explicitly, rather than guessed — that’s a design choice, not a gap we’re hiding.